

UNIVERSITY OF TORONTO MISSISSAUGA
CSC263 Summer 2025 Midterm CSC263 Data Structures and Algorithms
Duration: 120 minutes
Instructor(s): Peter Dixon
Test Aids: Your brain. You got this!

Please read the following guidelines carefully.

- Please **print** your name and UtorID above.
- **DO NOT** write anything in the QR code area.
- **DO NOT** open or turn over the test paper until the test has started.
- This test has **5** questions. There are a total of **10 pages, DOUBLE-SIDED**.
- Answer questions clearly and completely. Provide justification **unless explicitly asked not to**.

Question	Out of
Q1	4
Q2	10
Q3	4
Q4	4
Q5	10
Total	32

1. [4 marks] A giant social media company runs a platform in which users can create groups, assign moderator status, invite other users to groups they moderate, and send messages to users that share a group with them.

The company is making a tool for group creators that recommends users to invite based on shared groups. That is, it searches for users in the current group, finds ‘shared’ groups that lots of current-group users are in, and recommends users that are in the shared groups but not the current group.

There are several things that might affect runtime in this situation. When analyzing the runtime of the tool, which thing should n represent? Explain your decision in at most 2 paragraphs. (You do not have to write code. Just decide what to base the runtime on, and explain why. There are many possible answers here - you will be graded primarily on whether your justification demonstrates understanding of how to connect big-O runtime to real situations.)

2. [10 marks] We will use $\sim$ to denote the bitwise XOR operation. (See the cheat sheet for a definition, some useful $\sim$ properties, and an example of computing h_3 .) Let h_k be a hash function for BSTs whose nodes contain k -bit binary strings, defined recursively on the BST structure like so:

$$h_k(\text{node}) = \begin{cases} h_k(\text{node.left}) \sim h_k(\text{node.right}) & \text{node has 2 children} \\ h_k(\text{node.left}) & \text{node has only a left child} \\ h_k(\text{node.right}) & \text{node has only a right child} \\ h_k(\text{node.value}) & \text{node has no children} \end{cases}$$

The hash of the BST is $h_k(\text{root})$. You can assume that all node values are unique.

- (a) [2 marks] What is the table size for h ? Give a brief explanation.
- (b) [4 marks] Prove that there can be collisions between trees with different sets of leaves.
- (c) [4 marks]* Prove that h will collide whenever T_1 and T_2 have the same set of leaves. (Note: This is in my opinion the hardest question on the test. I recommend working on the other questions first. But do try for some partial credit here, at least.)

Additional space for 2c

3. [4 marks] * Give pseudocode for a combine-heap function that gets two **linked-list-based** heaps h_1, h_2 , and returns a single heap containing all nodes in h_1 and h_2 . (You may assume that h_1 and h_2 have a disjoint set of node priorities.)

4. **[4 marks]** Smeve eats lucky oatmeal for breakfast on test days. However, Smeve believes the oatmeal is only lucky once per weekday and once per numeric day. For example, if Smeve eats lucky oatmeal on Friday the 12th, Smeve can't eat lucky oatmeal on Tuesday the 12th or Friday the 4th.

This means Smeve has to plan carefully for test days. Smeve wants a data structure with one operation: `insert(weekday, numeric day)`. Insert should return `False` if the inserted day is the same weekday or date as any previously inserted day (and `True` otherwise), and be as efficient as possible.

Give pseudocode for the data structure.

5. [10 marks] Consider a modified BST that rebuilds itself when it gets too unbalanced. Specifically, if the length of the path from the root to any leaf is at least $\lceil \log n \rceil + 2$, the tree is collapsed into a list, then rebuilt into a nearly-complete BST. You can assume this is done in $O(n)$ time.
- (a) [4 marks] To quickly determine if the BST is severely unbalanced, what additional information should the BST be augmented with? Describe briefly how to modify insert to maintain the additional information. (Don't worry about delete.)
- (b) [2 marks] What is the **worst case** runtime of insert? Explain.
- (c) [4 marks]* Do an amortized analysis of the runtime of a sequence of inserts. (First identify a worst possible sequence of inserts and explain why it's the worst, then analyze its runtime.)

Extra space for 5c

Formulas

Triangular numbers: $\sum_{i=0}^n i = \frac{n(n+1)}{2}$

Geometric series: $\sum_{i=0}^n x^i = \frac{x^{n+1} - 1}{x - 1}$

Big-O ranking of functions (here $f(n) < g(n)$ means “ $f(n)$ is $O(g(n))$): $\log n < \sqrt{n} < n < n^k < 2^n < n!$

Runtimes

In all of these, n is the size of the list, heap, BST, or array. All times are worst-case. h is the height of the BST, which is $O(\log n)$ if using a self-balancing BST such as an AVL tree.

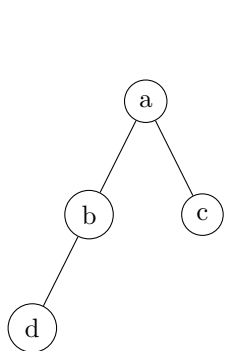
Heaps	
swap-up and swap-down	$O(\log n)$
build heap	$O(n \log n)$
build heap from array	$O(n)$
BSTs	
build bst	$O(n \log n)$
build bst from sorted array	$O(n)$
tree traversals (pre-order, in-order, post-order, level-order)	$O(n)$
search, insert, delete	$O(h)$
Arrays and lists	
sort a list	$O(n \log n)$
merge two sorted lists (m is the size of the second list)	$O(n + m)$
insert in array	$O(n)^*$
insert in linked list	$O(1)^*$
access array item at index	$O(1)$
access linked list item at index	$O(n)$
Hash tables	
add x to hash table	$O(n)^*$
look up x in hash table	$O(n)^*$
delete x from hash table	$O(n)^*$

Inserting in an array is $O(1)$ unless the array is full, and is $O(1)$ amortized using dynamically sized arrays.

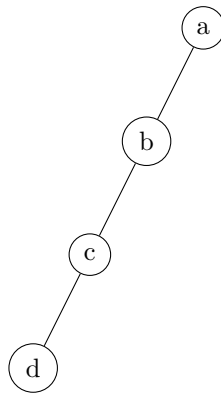
Hash tables are $O(1)$ average, assuming the hash function is good.

Nearly complete

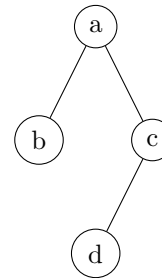
An n -node BST is nearly complete if its height is $\lceil \log n \rceil$ and all missing nodes are to the right.



Nearly complete



Not nearly-complete because the tree height is $> \lceil \log 4 \rceil$



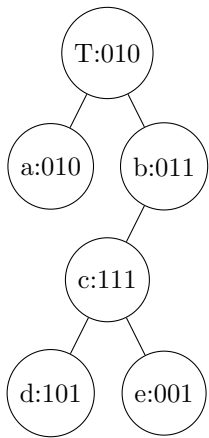
Not nearly-complete because a node with children (c) is right of a node without children (b)

Bitwise XOR

Bitwise XOR, written here as $\sim$, takes two same-length binary strings and XORs each bit. (XOR takes two True/False values and returns False iff the inputs are the same. We consider 1 as True and 0 as False.) For example, $0011 \sim 0110 = 0101$, because $0 \text{ XOR } 0 = 0$, $0 \text{ XOR } 1 = 1$, $1 \text{ XOR } 1 = 0$, $1 \text{ XOR } 0 = 1$.

Bitwise XOR is commutative and associative. That means $a \sim b = b \sim a$ and $(a \sim b) \sim c = a \sim (b \sim c)$.

Here is an example of computing h on a BST storing 3-bit binary strings. I'm labeling nodes in the format *name* : *value*, so I can refer to the nodes easily. It's easiest to compute h from the bottom up.



$$h(e) = 001$$

$$h(d) = 101$$

$$h(c) = h(d) \sim h(e) = 100$$

$$h(b) = h(c) = 100$$

$$h(a) = 010$$

$$h(T) = h(a) \sim h(b) = 110$$